# Laboratório Funcional · Aula 12

Gustavo Fumagalli Bandeira · RA 24011199-2 · Turma B
Gerado em 16/10/2026, 20:13:30 · tempo registrado no cronômetro: 45:26

## Preparação: Programar com funções matemáticas

**def area(r): return 3.14159 * r * r**

Pura

**def agora(): return datetime.now()**

Impura

**def total(lista): return sum(lista)**

Pura

**def registra(msg): print(msg)**

Impura

**def sorteia(): return random.randint(1, 6)**

Impura

**def poe(lista, x): lista.append(x); return lista**

Impura

**0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.**

O agora() é impuro porque depende do relógio: cada vez que chama, volta um horário diferente. O poe() é impuro porque altera a lista que recebeu com o append, ou seja, mexe em algo fora dela

## Prática 1: Funções puras e formas funcionais

**Previsão: O que as linhas 1, 4 e 7 vão imprimir?**

Linha 1: 1: A-1 A-2
Linha 4: 4: 14
Linha 7: 7: [7, 3, 9] [3, 7, 9]

**Saídas (original e com compor(g, f))**

```
Primeira Saída:
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
Segunda Saída:
1: A-1 A-2
2: True
3: 8
```

```
4: 18
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

**1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?**

Porque ela depende do contador global, que muda a cada chamada. Aí a mesma entrada dá resultados diferentes, e no teste você não sabe que valor esperar, depende de quantas vezes ela já foi chamada antes.

**1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.**

Compor(f, g)(4) = f(g(4)) = f(12) = 14. compor(g, f)(4) = g(f(4)) = g(6) = 18. Muda a ordem: no primeiro multiplica e depois soma, no segundo soma e depois multiplica.

**1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?**

O sorted(notas) cria uma lista nova e deixa a original quieta. O notas.sort() muda a própria lista. O sorted é o estilo funcional, porque não tem efeito colateral.

## Prática 2: Scheme: tudo é lista

**Previsão: (car '(A B C))**

A

**Previsão: (cdr '(A B C))**

(B C)

**Previsão: (cons '(A B) '(C D))**

((A B) C D)

**Saídas (original e sem o apóstrofo)**

```
Primeira Saída:
Output:
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B

Segunda Saída:
Output:
22
14
25
8
negativo
A
Backtrace:
```

```
           5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
   191:35  4 (_ #f)
   191:35  3 (_ #f)
   196:27  2 (_ #f)
   223:20  1 (proc #<directory (guile-user) 1008af098c80>)
In unknown file:
           0 (%resolve-variable (7 . A) #<directory (guile-user) 100…>)

ERROR: In procedure %resolve-variable:
Unbound variable: A
```

**2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?**

O apóstrofo diz pro Scheme "isso é só dado, não executa". Sem ele, o Scheme tentou rodar (A B C) como chamada de função, tratando A como função, e deu erro porque A não existe.

**2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.**

```
(car (cdr (cdr '(A B C)))), resultado: C.
Output:
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
C
```

## Prática 3: Recursão e recursão em cauda

**O que vi no passo a passo**

*não respondido*

**Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?**

Acho que no Scheme funciona e imprime 1000000. No Python acho que vai estourar a pilha e dar erro.

**Saída do Scheme**

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

**Saída do Python (as últimas linhas bastam)**

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
```

```
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

**3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?**

Na recursão comum cresce a pilha de chamadas esperando a volta, cada uma segurando uma multiplicação pendente. Na em cauda não cresce, porque a conta já vai no parâmetro e nada fica esperando.

**3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)**

O Scheme faz a otimização da chamada em cauda e reaproveita o registro de ativação, então a pilha não cresce. O Python não faz isso e ainda tem um limite de chamadas empilhadas (umas 1000), por isso deu RecursionError: maximum recursion depth exceeded.

## Prática 4: Mão na massa: três funções sobre listas

**Saída final (com os três ok)**

```
Primeira Saída:
Output:
tamanho: 0  (esperado 3)
soma-quadrados: 0  (esperado 14)
pares: ()  (esperado (2 4 6))

Segunda Saída:
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

**4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).**

```
(define (tamanho lis)
  (if (null? lis) 0 (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis) 0
      (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

**4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?**

A mais chatinha foi a pares, porque tem três casos. Nas três usei null? pra parar quando a lista acaba, car pra pegar o elemento atual e cdr pra chamar de novo com o resto.

## Prática 5: Haskell: padrões, compreensões e preguiça

**Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?**

O take 5 squares deve dar [0,1,4,9,16]. E o primeiro 42 (div 1 0) acho que dá 42, porque a função nem usa o segundo parâmetro.

**Saídas (original e com member 17)**

```
Primeira Saída:
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

Segunda Saída:
Output:
Time limit exceeded
```

**5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?**

Por causa da avaliação preguiçosa: o Haskell só calcula os elementos que precisa, então pega os 5 primeiros e para. E como primeiro ignora o segundo parâmetro, o div 1 0 nunca é calculado, aí não dá erro.

**5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)**

Ficou rodando pra sempre. O 16 está na lista, então a busca acha e para. O 17 não é quadrado, aí ele continua procurando na lista infinita sem fim. Pra corrigir, dá pra aproveitar que a lista é crescente e desistir quando passar do número: member2 b (m:x)

| m < b = member2 b x
| m == b = True
| otherwise = False

## Prática 6: Você já usa funcional

**Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?**

O mais10(5) deve dar 15 e [1, 2, 3]. E o map(vezes(5)) deve dar [5, 10, 15].

**Saída do JavaScript**

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

**Saída do Java**

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

**6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?**

É um fechamento: a função devolvida guarda uma ligação com o ambiente onde foi criada, então o n = 10 continua existindo enquanto o mais10 existir.

**6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?**

O compose aplica primeiro o g e depois o f: f(g(4)) = 14. O andThen aplica primeiro o f e depois o g: g(f(4)) = 18. O compose é o que corresponde ao compor(f, g) da Prática 1.

## Prática 7: O laço vira pipeline

**7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.**

```
int soma = nums.stream()
    .filter(n → n % 2 == 0)
    .mapToInt(n → n * n)
    .sum();
System.out.println(soma);
```

**7.2 Segundo exercício escolhido: cole o seu código reescrito.**

```
const aprovados = alunos
  .filter((a) ⇒ a.nota ⩾ 6)
  .map((a) ⇒ a.nome.toUpperCase());
console.log(aprovados.join(", "));
```

**Saídas das suas versões**

```
Saída EX1: 244
Saída EX2:
ANA, CARLA, DAVI
ANA, CARLA, DAVI, ANA, CARLA, DAVI
Saída EX3:
[50.0, 108.0, 80.0, 180.0]
```

**7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?**

A funcional é mais fácil de ler, porque fica tipo uma frase: filtra, transforma, soma. Mas pra depurar o laço é mais fácil, porque dá pra colocar um print no meio e ver cada valor.

## Fechamento: O que fica do paradigma funcional

**Uma função pura pode ler uma variável global que muda.**

Falso

**Em Scheme, a recursão em cauda não faz a pilha crescer.**

Verdadeiro

**O Python otimiza a recursão em cauda.**

Falso

**Em Haskell, [1 ..] só é calculada à medida que é usada.**

Verdadeiro

**filter, map e reduce dispensam laço e variável acumuladora.**

Verdadeiro

**Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.**

Verdadeiro

**8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?**

A do Haskell é mais fácil, numa linha só dá pra ler "gera a lista, eleva ao cubo, soma", sem ficar acompanhando variável mudando. Eu aceitaria ser 2x mais lento num sistema comum, tipo um site ou sistema de cadastro, onde clareza vale mais. Não aceitaria em coisa que precisa de muita velocidade, tipo jogo, sistema embarcado ou processamento pesado em tempo real.

**8.2 Em uma frase: o que mais surpreendeu você nesta atividade?**

Me surpreendeu dar pra trabalhar com uma lista infinita sem travar o programa.

**8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?**

*não respondido*